

Specifica del Progetto: Basi di Dati e Programmazione Orientata agli Oggetti

Anno Accademico 2024-25

1 Introduzione

Questo documento definisce la specifica estesa per un progetto congiunto di Basi di Dati e Programmazione Orientata agli Oggetti da svolgere dagli studenti del primo anno del corso di laurea in Informatica. Il progetto sarà svolto in gruppi di due o tre studenti. Ogni gruppo dovrà ideare, progettare e implementare un piccolo sistema informativo su un tema a scelta, sviluppando:

- un database in PostgreSQL
- un'applicazione Java con architettura a tre livelli:
 - Presentation layer (interfaccia)
 - Business logic layer (logica applicativa)
 - Persistence layer (gestione dati)

2 Panoramica del progetto

Scopo: realizzare un sistema informativo completo, esercitando tecniche di analisi dei requisiti, modellazione dati, progettazione della base di dati, progettazione OO e implementazione.

Risultato richiesto: database PostgreSQL funzionante e applicazione Java a tre livelli.

3 Scelta del tema

Selezionate un dominio reale a vostra scelta, sufficientemente complesso da includere:

- Uno schema concettuale con almeno 5 entità/classi per gruppi di due studenti, o 7 per gruppi di tre.
- Uno schema dei casi d'uso (Use Case diagram) con almeno 6 casi d'uso per gruppi di due studenti, o 8 per gruppi di tre.
- Almeno una generalizzazione
- Regole di business (vincoli semantici) non banali, che richiedano l'implementazione di almeno tre trigger.

Esempi:

- Gestione consegne di pacchi
- Prenotazione eventi
- Inventario e ordini
- Iscrizione e frequenza a corsi universitari

4 Obbiettivi di apprendimento

- Raccogliere e documentare requisiti funzionali ed eventualmente non funzionali.
- Modellare casi d'uso e dominio di interesse con opportuni diagrammi UML/ER.
- Definire operazioni ad alto livello e loro mappatura a metodi Java, eventualmente su dati persistenti.
- Progettare schema concettuale (ER/UML) e tradurlo in schema relazionale con vincoli
- Implementare regole di integrità tramite vincoli di base, trigger o logica applicativa.
- Applicare principi OOP (incapsulamento, ereditarietà etc.) in Java.
- Produrre pseudo-codice chiaro per operazioni critiche.
- Scrivere codice funzionante nonché script SQL di creazione e popolamento dati.

5 Documentazione

1. Requisiti in linguaggio naturale
2. Diagramma dei casi d'uso con descrizioni testuali a supplemento
3. Lista delle funzioni ad alto livello
4. Schema concettuale (diagramma ER o UML)
5. Catalogo delle regole di business (vincoli semantici)
6. Schema concettuale ristrutturato
7. Schema logico del database con vincoli di base
8. Diagramma dettagliato delle classi dell'applicazione (UML) con operazioni
9. Specifiche dettagliate delle operazioni principali (pseudo-codice)
10. Piano di implementazione dei vincoli sui dati (constraint, trigger, logica)
11. Codice Java completo (architettura a tre livelli)
12. Script SQL per creazione dello schema (DDL) e popolamento dati (DML) con dati di esempio.

Nota: Tutti gli elaborati da 1 a 10 devono essere inclusi in un unico PDF. Il codice Java e gli script SQL restano in file separati.

6 Istruzioni dettagliate

6.1 Specifica dei requisiti

- Scopo e ambito del sistema
- Attori primari
- Requisiti funzionali (elenco casi d'uso)
- Eventuali requisiti non funzionali

6.2 Diagramma dei casi d'uso e descrizioni

- UML use-case diagram con attori e casi d'uso
- Per ciascun caso d'uso:
 - Nome
 - Attore(i)
 - Precondizioni
 - Scenario principale
 - Flussi alternativi (se presenti)

6.3 Lista delle funzioni ad alto livello

Raggruppate per modulo (es. Gestione Utenti, Gestione Ordini):

- Nome funzione
- Descrizione breve
- Input/Output principali

6.4 Schema concettuale

Diagramma ER o UML con entità/classi, attributi, relazioni, cardinalità e gerarchie.

6.5 Regole di business

Tutti i vincoli che devono sussistere sui dati, inclusi quelli semantici che non possono essere rappresentati con vincoli di base; per esempio:

- “Un ordine deve riferirsi a un cliente esistente.”
- “Se il cliente ha tessera *oro*, il prezzo finale dell'acquisto deve essere scontato del 30% rispetto a quello calcolato dal carrello; se ha tessera *argento*, lo sconto sarà del 15%”.
- “La giacenza di un prodotto non può scendere sotto zero.”

6.6 Schema concettuale ristrutturato

Schema opportunamente ristrutturato per essere tradotto direttamente in schema relazionale.

6.7 Schema logico del database

Tabelle con colonne, tipi, PK, FK, NOT NULL, UNIQUE, CHECK.

6.8 Diagramma dettagliato delle classi dell'applicazione

Diagramma UML con package/pacchetti presentation, service, dao; classi, metodi (con segnatura), relazioni.

6.9 Specifiche dettagliate delle operazioni

Per i casi d'uso più critici:

- Pseudo-codice
- Segnatura dei metodi
- Logica di business
- Operazioni su DB

6.10 Implementazione dei vincoli sui dati

Per ogni regola di business (vincolo semantico), indicare se implementata tramite:

- Constraint SQL (PK, CHECK, FK)
- Trigger (specificandone eventi, condizione e azione)
- Validazione in applicazione (classe/metodo e a quale dei tre livelli)

7 Codice

7.1 Applicazione Java

- Gestione della persistenza con JDBC
- Architettura a tre livelli

7.2 Script SQL

- Script DDL (creazione schema)
- Script DML (popolamento con dati di esempio)

8 Linee guida per la consegna

- Compilare tutti gli elaborati (documenti 1–10) in un unico file PDF. Il documento non deve essere una mera sequenza degli elaborati elencati nella sezione 5, ma una relazione sul progetto nella sua interezza.
- Organizzare la struttura di file e cartelle all'interno del file Zip come specificato di seguito
- Verificare che il codice Java compili e gli script SQL si eseguano senza errori.

Struttura del contenuto del file Zip (si omette la terza matricola per i gruppi di due studenti):

```
matricola1_matricola2_matricola3-prog_BD_P00.zip
├─ matricola1_matricola2_matricola3-prog_BD_P00/
│   ├── partecipanti/
│   │   └─ partecipanti.txt (file di testo con nome, cognome e matricola dei partecipanti)
│   ├── pdf/
│   │   └─ matricola1_matricola2_matricola3-prog_BD_P00.pdf
│   ├── java/
│   │   └─ (codice sorgente, eventualmente diviso in sottocartelle)
│   └─ sql/
│       ├── schema.sql (creazione tabelle)
│       └─ dati.sql (popolamento)
```

9 Discussione ed esame

- I gruppi sono formati da due o tre studenti.
- La discussione del progetto avviene in una sessione orale ad ogni appello, secondo calendario.
- Quando un gruppo decide di discutere il progetto ad un certo appello,
 - tutti i suoi elementi devono presentarsi insieme alla discussione;
 - il progetto deve essere consegnato almeno tre giorni prima della data della discussione.
- Ogni progetto presentato riceverà un voto per Basi di Dati I e uno per Programmazione Orientata agli Oggetti, che potranno essere diversi essendo essi riferiti a diversi aspetti del progetto presentato.
- Ogni studente potrà sostenere l'esame scritto prima o dopo la presentazione del progetto.